

Testra

Complete Technical Reference

An AI system that reads a web codebase and writes runnable end-to-end test suites for Playwright, Cypress, or Selenium — grounded in selectors extracted from the user's real source code, and verified against it before they ship.

Repository	github.com/Krishna417211/testgen-ai
Branch	feat/admin-console-and-account-flows
Stack	Python 3.11+ / FastAPI · React 18 / TypeScript / Vite
Scale	~12,000 lines backend (32 files) · ~9,000 lines frontend (61 files)
Surface	45 HTTP endpoints · 8 SQLite tables · 38 config settings · 650 tests
Generated	19 July 2026

Contents

1. What Testra Is — the product in one page
2. System Architecture — components and data flow
3. The Request Lifecycle — analyze → stream → generate
4. Step Progress — streaming the long POSTs
5. The Two-Agent Pipeline — FilterAgent and WriterAgent
6. The Trust Pipeline — grounding, self-heal, fragility
7. The Scaffold — turning tests into a runnable suite
8. The LLM Router — tiered, multi-provider rotation
9. The File Extractor — deterministic pre-filtering
10. Authentication and Sessions
11. Email, Verification, and One-Time Secrets
12. The Admin Model
13. Quota and Billing
14. The Security Scanner
15. GitHub Integration and Publishing
16. The Data Model — 8 tables
17. Complete API Reference — 45 endpoints
18. Frontend Architecture
19. Configuration Reference — 38 settings
20. Testing and CI
21. Deployment — AWS, OIDC, and the rollout
22. Known Limitations
23. Open Findings — issues identified in review
24. Appendix — full file inventory

1 · What Testra Is

Testra takes a GitHub URL (or an uploaded ZIP) and produces a complete, runnable end-to-end test suite. The product promise is on the landing page: *"Paste a GitHub URL. Get production-ready E2E tests in seconds."*

The problem it solves

Asking a general-purpose LLM to "write E2E tests for my app" produces tests full of invented selectors — `#login-button` where the real element is `[data-testid="submit"]`. The tests look right and fail immediately. Testra's entire architecture is a response to that single failure mode: **ground the model in selectors that provably exist in the source, then verify what it produced against that source and flag anything it invented.**

What it produces

- **Page Object Model classes** — one per key page, not flat scripts
- **Spec files** grouped by page or flow, with positive and negative cases
- **A runnable scaffold** — dependency manifest, framework config, CI pipelines and README, all templated (§7)
- **A trust report** — per-selector provenance, live-DOM verification, and a durability grade (§6)
- **Selector warnings** — anything not verifiable against source is marked [⚠](#)

Supported targets

Framework	Languages
Playwright	JavaScript, TypeScript, Python
Cypress	JavaScript, TypeScript
Selenium	Python, Java

Input stacks it recognizes: React, Vue, Next.js, Nuxt, Angular, Svelte on the SPA side; Django, Flask, FastAPI, Rails, Laravel plus their template engines on the server-rendered side.

Feature surface beyond test generation

The product has grown past its original single purpose. It now includes:

- **A trust pipeline** — selector grounding against source *and* the live DOM, a self-heal loop, and fragility grading
- **Live step progress** — the long POSTs stream a flowchart of what is running (§4)
- **Repo publishing** — push a generated suite into a brand-new GitHub repo in one commit
- **Passive security scanning** — grade a deployed URL's production configuration
- **Accounts** — email/password, GitHub OAuth, *Google OAuth*, email verification, login OTP, password reset
- **Plans and quota** — free tier metered per calendar month, unmetered Pro, Stripe-backed

- **A user dashboard** — activity feed, per-day charts, published repos
- **An admin console** — a standalone portal for user management, plan/quota override, provider health, and platform analytics

Design signature. Throughout this codebase the docstrings record *why* a decision was made and what breaks otherwise — not what the code does. That reasoning is preserved verbatim in this document wherever it explains a non-obvious choice, because in several cases the "obvious" alternative is a security or correctness bug.

2 · System Architecture

FRONTEND – React 18 / TypeScript / Vite

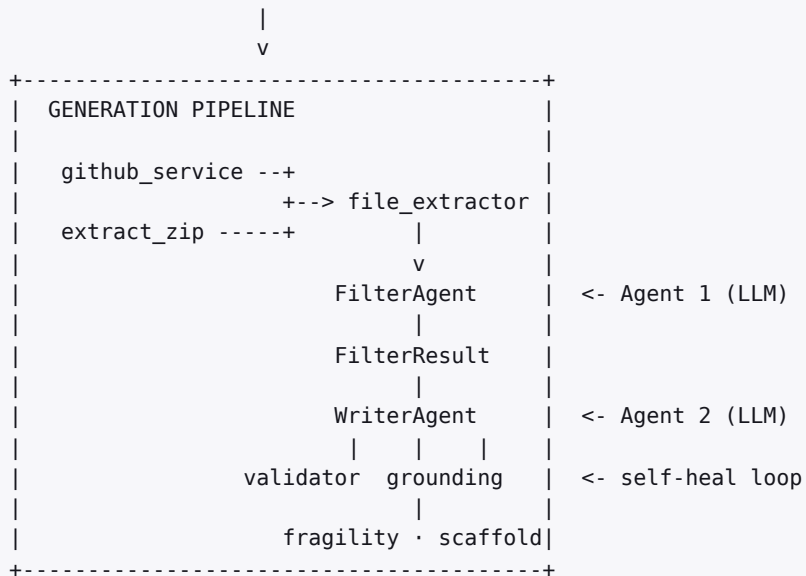
19 routes · 29 components · AuthContext · ProtectedRoute

REST + SSE + NDJSON (Bearer token, or ?token= for EventSource)

API – FastAPI (main.py, 2116 lines, 45 routes)

request-id middleware · CORS · rate limiting · JSON logging

auth	quota	admin	scanner	publish	verificat.
+OAuth	+billing				+mailer
+-----+-----+-----+-----+-----+					



LLMRouter
tier-aware: FREE / PRO
round-robin keys
429 -> cooldown
provenance recording

JobStore (SQLite)
8 tables, stdlib
survives restarts
TTL pruning

Layer responsibilities

Layer	Modules	Owns
API	main.py, models/schemas.py, ratelimit.py, progress	HTTP surface, validation, rate limits, correlation IDs
Identity	auth, github_oauth, google_oauth, verification, mailer, admin	Who the caller is and what they may do
Commerce	quota, billing	Metering, plans, entitlement grants
Pipeline	file_extractor, filter_agent, writer_agent, validator, scaffold	Turning a repo into a runnable suite
Trust	grounding, fragility, importance_model, benchmark	Proving the suite is worth running
Providers	llm_router	Reaching an LLM despite rate limits
Features	security_scanner, git_publisher, github_service	Scanning, publishing, fetching
Persistence	store	Every durable fact, in one SQLite file
Cross-cutting	config, logging_config	Typed settings, structured logs

No external infrastructure. Persistence is stdlib `sqlite3` ; rate limiting is in-process; there is no Redis, no queue, no ORM. The system runs from a single process with no services to provision. The cost is documented in §22: rate limits are per-worker, so multi-worker deployment weakens them.

3 · The Request Lifecycle

Test generation is a three-call flow, deliberately split so the expensive LLM work is visible while it happens.

Step 1 — Analyze `POST /api/analyze`

1. `parse_github_url` splits the URL into owner / repo / branch / subpath
2. `GitHubService.get_file_tree` pulls the whole tree in one Git Trees API call (`recursive=1`)
3. `fetch_files` pulls contents concurrently — `raw.githubusercontent.com` for public repos (no rate limit), the Contents API for private
4. `FileExtractor.extract` filters, scores, and budgets the files — no LLM, fast and free
5. `FilterAgent` asks the LLM for a structured project analysis
6. The job is persisted in SQLite; `job_id` + analysis returns to the client

The alternative entry point, `POST /api/upload-zip`, replaces steps 1–3 with `extract_zip` (in-memory, handles GitHub's nested ZIP layout) and is otherwise identical.

Both stream their progress as NDJSON rather than returning a single JSON body — see §4.

Step 2 — Stream `GET /api/stream/{job_id}` (optional)

Server-Sent Events stream Agent 2's output token by token. The raw output is cached onto the job row, so a subsequent generate call reuses it rather than paying for a second LLM call — the `pregenerated_raw` parameter on `WriterAgent.run`.

Why SSE takes a query-param token. The browser `EventSource` API cannot set custom headers, so it cannot send `Authorization: Bearer . auth._extract_token` therefore accepts `?token=` as well. This is a deliberate, documented concession to a browser limitation, not an oversight.

Step 3 — Generate `POST /api/generate/{job_id}`

1. `require_quota` authenticates and reserves a generation credit
2. Ownership is re-checked — a job belonging to another account is 403, a missing one 404
3. `WriterAgent.run` plans the file list, then generates each file
4. `validator.validate_files` syntax-checks the output; failures drive the self-heal loop
5. Selectors are grounded against source and, when a URL is supplied, the live DOM (§6); unverified ones are re-prompted, then flagged
6. `scaffold` adds the manifest, config, CI and README that make the suite runnable (§7)
7. On success the lease commits; **on failure it refunds** — a crashed run costs no credit

Failed runs are recorded, not dropped. From `main.py`: *"from the dashboard's side a run that vanished and a run that never happened look identical, and the user who watched this spin for a minute deserves to see it happened."* Failures are written to `generations` with `status='failed'` and surface in the admin console's `recent_failures` view.

4 · Step Progress NEW

services/progress.py — 191 lines

`/api/analyze`, `/api/upload-zip`, `/api/scan` and `/api/publish-zip` each do 10–60 seconds of work behind a single request, and the client had no way to know which part was running. This module streams the answer.

Each endpoint emits **NDJSON**: one `{"type": "step"}` line per phase as it starts and finishes, then a single `result` or `error`. The client (`FlowPipeline`) lights up a flowchart from it, so the wait shows which phase is running and what it found:

```
{"type": "step", "id": "extract", "state": "done",  
  "detail": "18 of 240 files kept · React"}
```

Why NDJSON and not SSE. All four endpoints are POSTs and two carry a multipart upload, while `EventSource` can only issue a GET with no headers and no body. Streaming the POST's own response body keeps one request, the `Authorization` header and the file upload exactly as they are — only the response *shape* changes.

The error split this forces

The status line goes out before the work starts, so a failure mid-flight can no longer *be* an HTTP status. That splits error handling in two, and the split is load-bearing:

- Anything decidable up front — a malformed URL, a missing GitHub token, an exhausted quota — **must** be raised before the first byte is yielded, so it stays a real 400/403/402 and the client keeps reacting to it as it always has (`QuotaExceededError` → the upgrade modal).
- Anything that can only fail once the work is underway is reported in-band as `{"type": "error", "status": N, "detail": ...}` and re-raised on the client (`raiseApiError` in `utils/api.ts`) as the same exception the non-streaming call would have thrown.

Step ids are a cross-language contract. They are shared with `frontend/src/flows.ts`. A drifted id is silent in the browser — the card simply never lights — so `Progress` rejects an unknown id rather than emitting an event nothing listens for.

5 · The Two-Agent Pipeline

Agent 1 — FilterAgent (agents/filter_agent.py, 269 lines)

Runs in two phases.

Phase 1a — local extraction (no LLM)

`FileExtractor.extract` does the heavy lifting deterministically. See §9.

Phase 1b — LLM project analysis

Sends a compact file-tree summary and asks, in JSON mode, for a `FilterResult`: project summary, key pages with test priorities, key components, routes, and testing challenges.

The `FilterResult` contract — the pipeline's central data structure:

Field	Type	Meaning
project_summary	str	Human-readable summary shown in the UI
key_pages	list[dict]	{path, description, test_priority, suggested_tests}
key_components	list[dict]	{path, description}
routes	list[str]	Routes detected in routing code
framework	str	Detected stack
testing_challenges	list[str]	Obstacles, plus extractor warnings
files	dict[str,str]	The filtered source — passed whole to Agent 2
total_tokens, file_count	int	Extraction accounting

Two anti-hallucination defences, both learned the hard way.

- NoTestableUIError**. If extraction yields zero browser-renderable files, the pipeline stops rather than calling the LLM. The docstring records why: *"with an empty file tree the LLM has nothing to describe, and it answers by echoing the shape of the example in the prompt — inventing a login page, a /dashboard route and a JWT flow for a project that has none. Failing loudly beats a confident fiction."*
- Placeholder-only schemas**. The prompt's JSON schema uses `<angle-bracket>` placeholders, never realistic samples. Again from the source: *"An earlier version illustrated the shape with 'src/pages/Login.tsx', routes ['/', '/login', '/dashboard'] and a JWT/infinite-scroll challenge — and the model reproduced those verbatim for projects that contained none of them. Describe the shape; never supply content that is plausible enough to copy."*

The prompt also carries explicit grounding rules that override everything else: every path must appear verbatim in the file tree, every route must be traceable to shown code, and empty arrays are correct where padded filler is a failure.

Agent 2 — WriterAgent (agents/writer_agent.py, 1113 lines)

File-by-file generation

The original design asked for the whole suite as one JSON blob and hit the output-token cap on large repos — silently truncating. The current design is two-stage:

1. `_plan_files` — asks only for a manifest (filenames + roles). A tiny response that cannot truncate. Capped at 12 files (`writer_agent.py:872`).
2. `_generate_one_file` — generates each file's contents in its own call, so every response stays well under the cap.

`_generate_single_shot` survives as a fallback when planning yields nothing.

Selector grounding

`_extract_available_selectors` (`writer_agent.py:1009`) mines the real source for `data-testid`, `data-cy`, `id`, `name`, and `class` values, and injects them into the prompt under the heading *"prefer these; anything else is a guess."* The model is instructed to mark any selector it must invent with `⚠ WARNING: Selector may need verification`.

What the model produces is then *checked* rather than trusted — see §6.

The self-heal loop

Two independent checks feed regeneration:

- **Syntax** — `services/validator.py` performs a real syntax check where a toolchain exists (`py_compile` for Python, `node --check` for JavaScript) and a structural check (bracket balance, non-truncation) otherwise.
- **Grounding** — every selector that failed to resolve is sent back with the real anchors that *do* exist, so the fix is a grounded substitution, not another guess.

Validation counts only what it actually checked. `FileValidation` tracks whether anything was verified at all. Config and docs pass by default, so counting them inflates the reported number: *"8/8 files passed syntax validation"* reads as *eight checked files when two of them were a README and a YAML nobody parsed*.

Failure honesty

When a single file fails to generate, the exception is *not* swallowed. From the source: *"Swallowing this used to turn 'the LLM died mid-suite' into a silent gap: the file vanished, the run still looked successful, and CI got pushed against a suite that couldn't run."* Failed filenames are recorded on `_failed_files` and surfaced.

6 · The Trust Pipeline NEW

The failure mode this product exists to prevent is a suite that looks perfect and breaks on the first run because the model invented a selector. Four components attack that, and **none of them execute the generated code** — there is no sandbox (see `services/validator.py`); they reason about the code and the app statically.

Grounding — does every selector resolve to something real?

`services/grounding.py` — 349 lines

From the module docstring: *"This is the trust spine of the product. An LLM will happily write `page.locator('#submit-btn')` for a button whose id is actually `submit`, and the suite looks perfect until it is run. We refuse to ship that unexamined."*

There are two ground truths, and they answer different questions:

Ground truth	What it proves	Availability
Source grounding	The selector exists in the repo's markup. Carries <i>provenance</i> — the file and line the element was declared on, so the UI can say "this came from <code>LoginForm.tsx:42</code> " instead of asking the user to trust us.	Any repo
DOM grounding	The selector matches an element on the <i>deployed</i> page right now. Stronger evidence — it is what the browser will actually see.	Only when the user supplies a URL they own

Live HTML is fetched through the security scanner's SSRF-guarded, non-executing path (`validate_target` + `_redirect_guard` , §14) — never executing anything, only reading the DOM.

Both are built the same way on purpose. Extract the set of anchors that exist (ids, test-ids, classes, form names, ARIA roles, accessible labels, link/button text), then check each generated selector's key against that set. A selector verified against source and one verified against the live DOM are judged by the same rules.

No CSS engine is needed: the selectors we emit reduce to a handful of anchor kinds, and membership is the question "would `querySelector` find this".

Fragility — which selectors will break *later*?

`services/fragility.py` — 262 lines

From the docstring: *"The failure that makes people abandon a generated E2E suite is not a selector that is wrong today — validation catches those. It is a selector that is right today and brittle tomorrow: `div > div:nth-child(3) > button` matches now and breaks the first time someone adds a wrapper."*

A weighted linear scorer over named, inspectable features — positional depth, hashed classes, absolute XPath, versus stable contracts like `data-testid/role` — predicts break-risk 0.0 (rock solid) → 1.0 (will break), and grades the suite A–F.

Why a feature model and not an LLM call. *"Brittleness is a small, stable, explainable function of a selector's shape ... That is exactly what a linear scorer over named features does well and cheaply, with no token cost and a reason attached to every score."* The weights are deliberately `data ( FEATURE_WEIGHTS )`, not code, so the benchmark harness can re-fit them from real pass/break data without touching the logic.

The score is named as a *risk* estimate — never dressed up as a probability we measured.

The importance model — a learned file scorer

`services/importance_model.py` — 226 lines

`file_extractor.score_file` ranks files with hand-written rules: a router is a 10, a page is a 9, a util is a 4. Those are good priors but frozen — they cannot notice that in *this* codebase the interesting behaviour lives in `features/` and the `pages/` are thin shells. This module learns that distinction from data.

It is a logistic-regression classifier implemented from scratch — no numpy, no sklearn — for three stated reasons:

1. **It's honest about being ML.** Every weight is a named coefficient on a named feature, fit by gradient descent and written to JSON you can read.
2. **It has no dependencies.** Adding a training stack for one small model would be a poor trade.
3. **It's re-fittable from real outcomes.** The bootstrap label is weak supervision; the same `fit` re-trains on real signal once the self-heal loop and benchmark harness have produced it.

It degrades safely: with no trained artifact on disk, callers fall back to the heuristic `score_file`, so importing it changes nothing until a model is trained.

The benchmark harness — the numbers, over many repos

`services/benchmark.py` — 198 lines

"A demo convinces one person; a benchmark convinces a room." Runs grounding and fragility over a set of (repo, generated-suite) cases and reports:

- **Verified-selector rate** — of every selector the model wrote, the fraction that provably resolves to a real element. This is the stand-in for "does it work": because suites are never executed, the harness reports the thing it *can* prove rather than an invented pass rate.
- **Fragility grade** — so a suite can't score well by grounding everything to `nth-child`.

The harness is honest about denominators: a case that grounded nothing counts as a case with no measurable rate, not a free 100%. Run with `python -m services.benchmark cases.json`.

All of this surfaces in the frontend `TrustPanel`: the live-DOM badge, the per-selector provenance, and the durability grade — each a real measurement, never dressed up as a prediction that the suite passes.

7 · The Scaffold NEW

agents/scaffold.py — 933 lines

The writer agent's job is page objects and specs. Everything that turns those files into a suite you can actually `npm install && npx playwright test` — the dependency manifest, the framework config, the CI pipeline, the README — is boilerplate with one correct answer per framework. It is built here, from templates, rather than asked of the model: *"the LLM adds nothing to a package.json and every token it spends on one is a token it can truncate."*

Two invariants this module exists to enforce

1. A suite ships its own dependencies. Nothing here ever emitted a `package.json` or a `requirements.txt`, while the CI it emitted ran `npm ci` and `pip install -r requirements.txt`. *Those pipelines could not pass on any repo:* there was no manifest to install and no lockfile for `npm ci` to read. The install step and the manifest are now written together, in this file, so they cannot drift apart again.

2. The suite lives in its own directory. Everything lands under `SUITE_DIR` (`e2e/`) instead of the repo root. A React app already has a `package.json` at the root, and writing ours there would silently destroy the app's own manifest — *the suite must never be able to damage the repo it is testing.*

Reaching the app under test

`BASE_URL` wins, then a local `webServer`. One variable points the same suite at staging or prod; unset, the config boots the app so a fresh clone and CI both work with no arguments.

Base URLs are normalized to a trailing slash and tests navigate with slashless relative paths — the only combination under which a sub-path deployment (a GitHub Pages project site) stays addressable.

8 · The LLM Router

services/llm_router.py — 948 lines

The router exists so that free-tier rate limits never surface to the user. Its strategy:

- 1. Try providers in priority order — *which order depends on the caller's tier*
- 2. Within a provider, rotate round-robin through every configured key
- 3. On 429 or error, back off exponentially, then move to the next key or provider
- 4. Track exhausted keys per reset window and auto-restore after cooldown
- 5. Stream live provider status to the frontend over SSE
- 6. Record *provenance* — which model actually produced each output

Tiers NEW

The router is now tier-aware. `MODELS` maps `(Provider, Tier)` → `ModelSpec`:

Provider	FREE tier model	PRO tier model	Max output	Cooldown
Gemini	gemini-2.0-flash	gemini-2.0-flash	8192	60s
Groq	llama-3.3-70b-versatile	llama-3.3-70b-versatile	4096	30s
Claude	claude-haiku-4-5	claude-opus-4-8	4096 / 16000	120s

Key env prefixes are unchanged: `GEMINI_API_KEY_1..10`, `GROQ_API_KEY_1..10`, `ANTHROPIC_API_KEY_1..10`.

Only Anthropic's rows differ between tiers, *"because that is where a materially stronger model is actually available to us: Gemini and Groq stay on their free-tier models and serve as capacity, not as the quality story."*

The Pro `ModelSpec` carries three extra flags, each for a concrete reason recorded in the source:

Flag	Why
<code>accepts_temperature=False</code>	Opus 4.7+ reject temperature/top_p/top_k outright (400). Haiku accepts them.
<code>adaptive_thinking=True</code>	<i>"most of what the Pro tier is actually buying: the model reasons about the repo before writing selectors instead of pattern-matching to a generic suite."</i>
<code>effort="high"</code>	<code>budget_tokens</code> is removed on 4.7+ — depth is set with effort, not tokens.

Priority order, per tier

Tier	Order	Rationale
FREE	Gemini → Groq → Claude	Capacity-first — Gemini's 1M context swallows big repos, Groq is fastest
PRO	Claude → Gemini → Groq	Quality-first — <i>"paying for Opus and then serving the request from Flash because Flash was listed first would make the upgrade a lie"</i>

Pro still falls back rather than failing. A Pro user who would otherwise get nothing gets a free-tier-quality suite — *and the provenance report says so.*

Key concepts

Type	Role
APIKey	One credential. <code>is_available</code> , <code>mark_exhausted</code> , <code>record_success</code> .
ProviderState	All keys for one provider. <code>next_available_key</code> round-robins, skipping exhausted.
ModelSpec	The model, output cap, and per-model API quirks.
ProviderCall	One successful call made while producing a user-visible output — provider, model, context, tokens.
LLMRouter	The engine. Thread-safe via <code>asyncio</code> locks — safe under concurrent FastAPI requests.
<code>is_hard_quota</code>	Distinguishes "out of credit / daily quota burned" from a transient 429.

Exceptions

- `RateLimitError` — 429 or signalled quota exhaustion
- `ProviderError` — non-rate-limit failure (5xx, malformed response)
- `AllProvidersExhausted` — every provider and key tried and failed

Per-request status isolation. Status callbacks are scoped with a `ContextVar` keyed to the current `async` task, so concurrent SSE streams never cross-talk. `subscribe_status` registers a callback for the current task only. The same mechanism scopes provenance recording.

Exhaustion is an outage, not an upsell. When every key is hard-blocked, capacity is gone for everyone — Pro subscribers included. `quota.py` records the reasoning: *"Upselling at that moment would be selling something we cannot deliver."* That path stays a plain 503; only genuine per-user limits return 402.

Observability

`get_status` feeds the public status panel. `get_key_health` feeds the admin console with per-key state, including which env var to rotate when one goes bad. `get_call_log` exposes recent calls; `_log_call` records

provider, key, context hint, latency, model, and token usage per call.

Docstring drift. The module header still reads *"Try providers in priority order (Gemini → Groq → Claude)"* and *"All providers are FREE tier — no credit card required."* Both predate the tier system and are now wrong for Pro. Recorded in §23.

9 · The File Extractor

services/file_extractor.py — 658 lines

The extractor is where most of the intelligence lives, and it costs nothing — no LLM call. Its stated core insight: *"we only want files that render in the browser. Backend logic, DB migrations, server configs — all noise."*

The scoring system

Score	Category
10	Router files (App.jsx, router/index.js) — always included
9	Page components (pages/, views/, screens/)
8	Top-level layout components
7	Form and auth components
6	Shared UI components
5	Config files (package.json, vite.config.js)
4	Utility components
1	Everything else UI-adjacent

These are frozen priors. `services/importance_model.py` (§6) refines them from data when a trained artifact is present.

Budget behaviour

Files are admitted in score order until `token_budget` (default **600,000**) is reached. Crucially, low-value files are **summarized rather than dropped** — `_summarize_file` keeps only the testing-relevant parts: element IDs, class names, form fields, button text, route definitions. Token estimation is `len(content) // 4`, a rough 4-chars-per-token heuristic.

Other responsibilities

- `detect_framework` — SPA frameworks, server-rendered backends, template engines
- `detect_monorepo` — identifies sub-packages
- `find_manifest` — shallowest file of a given name, at any depth
- `extract_zip` — in-memory ZIP handling, including GitHub's nested layout
- `filter_for_push` — what belongs in a fresh repo, with a 1 MB per-file cap (`PUSH_MAX_FILE_BYTES`)

10 · Authentication and Sessions

services/auth.py — 208 lines

Sessions are **opaque server-side tokens**, stored in SQLite, not JWTs. The reason is stated plainly: so they are revocable. A JWT cannot be un-issued; a row can be deleted.

Token acceptance

`_extract_token` accepts `Authorization: Bearer` or a `?token=` query parameter. The latter exists solely because `EventSource` cannot set headers (§3).

Function surface

Function	Role
<code>hash_password / verify_password</code>	Password hashing
<code>validate_email / validate_password</code>	Input rules
<code>encrypt_secret / decrypt_secret</code>	Fernet encryption for stored GitHub tokens. No-op if <code>SESSION_SECRET</code> is unset; legacy plaintext (no prefix) reads back as-is.
<code>create_login_session</code>	Mints an opaque token
<code>public_user</code>	Strips secrets before a user is sent to the client
<code>is_admin</code>	Whether the account holds the admin role
<code>require_user</code>	The FastAPI dependency — token → user, or 401

`require_user` returns a context dict: `{user, session, session_id, user_id}`.

Session lifetime

`touch_session` slides expiry forward on use — *an idle timeout, not a fixed one*. `delete_sessions_for_user` revokes every session for an account, optionally keeping the current one (the "log out everywhere else" behaviour).

OAuth — two providers

GitHub (services/github_oauth.py, 93 lines)

1. `authorize_url()` — send the browser to GitHub's consent screen
2. `exchange_code()` — swap `?code` for an access token
3. `get_user()` — resolve login, name, avatar

Lets a user authorize repo creation without pasting a Personal Access Token. Requires `GITHUB_CLIENT_ID` / `GITHUB_CLIENT_SECRET` .

Google (`services/google_oauth.py`, 108 lines) NEW

Identical in shape — `authorize_url()` → `exchange_code()` → `get_user()` reading the OpenID userinfo (`sub` , email, name, picture).

Google is only ever an identity here. Unlike GitHub it grants no repo access, *"so the session it produces carries no provider token, and the account it creates is email/password-less until the user sets one."*

Google will only redirect back to an `https://` callback on a real hostname — a bare IP or plain http is rejected at registration — so this needs the site reachable over HTTPS on a domain (§21).

The returning user is matched by `users.google_id` , Google's stable OpenID subject — mirroring `github_id` , so a user who later changes their email still resolves to the same account.

Rate limiting (`ratelimit.py`, 57 lines)

In-memory per-IP sliding windows, applied as FastAPI dependencies. Used via `Depends(rate_limit(...))` rather than a decorator — the module notes this is deliberate, because *unlike slowapi's decorator, it works correctly on routes with path params*.

Limiter	Budget	Applies to
<code>analyze_limiter</code>	20 / 60s	Repo analysis, ZIP upload, scan, publish, repo delete, most auth routes
<code>generate_limiter</code>	10 / 60s	Generation and streaming
<code>email_limiter</code>	6 / hour	Anything that sends mail
<code>otp_limiter</code>	20 / 10min	OTP submission attempts

The email limiter is deliberately much tighter than the rest: *"each call puts a message in someone else's inbox, so an unlimited one is a free mail-bombing service pointed at any address an attacker names, and it burns the relay's reputation and sending quota along the way."* The OTP limiter complements the per-challenge cap in `verification.py` — it stops someone parallelising guesses across many freshly-minted challenges from one host.

The limiters only work if uvicorn is told to trust the proxy. They key on `request.client.host` , but uvicorn only trusts `X-Forwarded-For` from `127.0.0.1` by default. Behind Caddy every request arrived wearing the proxy's container IP, so *all clients shared one bucket*: six signup/OTP emails an hour for the whole site, and any anonymous `/api/scan` could exhaust the login bucket and lock everyone out of sign-in.

The fix lives in `backend/Dockerfile` : `--proxy-headers` with `--forwarded-allow-ips` scoped to private ranges only — `127.0.0.1,10.0.0.0/8,172.16.0.0/12,192.168.0.0/16` . Never `"*"` , which would take the client-supplied left-most hop and let anyone forge their IP.

11 · Email, Verification, and One-Time Secrets

The mailer (services/mailer.py, 112 lines)

Transport only, over `aiosmtplib`. Async so a send never blocks the event loop the way `stdlib smtplib` would. `_assert_header_safe` rejects CR/LF in anything that becomes a header — header-injection defence.

Two exceptions distinguish the failure modes: `EmailNotConfigured` (no relay set up) and `EmailDeliveryError` (relay present but refused or dropped).

Dev without SMTP. With no relay configured outside production, the mailer *logs* the message instead of sending. The full signup → verify → OTP flow runs locally with no mail account. In production an empty relay is **refused rather than skipped**, so the checks cannot silently disable themselves.

Verification (services/verification.py, 253 lines)

This module carries the most carefully reasoned threat model in the codebase.

Why the two kinds of secret are stored differently.

Link tokens (verify, reset) are 256-bit random. *"Guessing one is not a thing that happens, so a plain SHA-256 is enough to keep the database from being a list of working links: it's preimage-resistant, and there's no dictionary to run against a value with that much entropy."*

The login OTP is six digits — a million possibilities, which a GPU chews through instantly. It therefore gets **bcrypt**, not SHA-256.

The asymmetry is the point: hash strength is chosen against the entropy of what is being hashed, not applied uniformly.

Function	Role
send_verification_email / consume_verification_token	Signup confirmation
send_password_reset_email / consume_reset_token	Password reset
start_login_challenge / verify_login_challenge	Login OTP, issued only after the password check passes
_new_code	6-digit code from a CSPRNG
mask_email	ada@example.com → a**@example.com

Two more deliberate choices:

- `InvalidCode` is one exception for *wrong, used, and expired*. *"Telling an attacker which is a free oracle."*
- `_new_code` uses `secrets`, never `random` — *"the latter's Mersenne Twister is reconstructible from past outputs."*

TTLs and limits

Setting	Default
email_verify_ttl_seconds	86,400 (24h)
password_reset_ttl_seconds	3,600 (1h)
login_otp_ttl_seconds	600 (10m)
login_otp_max_attempts	5 per challenge

12 · The Admin Model

services/admin.py — 57 lines. The smallest security-critical module, and the most opinionated.

How someone becomes an admin

By being listed in `ADMIN_EMAILS` with a **verified** address. That is the entire mechanism. There is deliberately no role column, no grant table, and no promote-user endpoint.

The reasoning, verbatim from the source:

- privilege can't be escalated by writing to the database, because the grant isn't in the database;
- there is no privileged-write endpoint to find a bug in, because there is no endpoint;
- a stolen admin session can act as that admin until it expires, but cannot mint a second admin.

The trade is explicit: changing the admin list is a config change plus a restart — *"the right friction for the one role that can read every account."*

Two rules the module exists to enforce

1. **The address must be verified.** With `REQUIRE_EMAIL_VERIFICATION=false` a stranger could sign up as an admin's address and be issued a session — so the allowlist alone is not proof of who holds the address.
2. **Non-admins get 404, not 403.** *"A 403 confirms the console exists to someone who already has a session; to everyone else it should look absent."*

The client's `user.is_admin` decides whether the nav item renders and **nothing else** — every route re-derives the role server-side via `require_admin`.

A standalone console NEW

The admin surface is no longer a section of the product. It is a separate portal behind its own sign-in (`pages/AdminLogin.tsx`) with its own shell (`components/AdminLayout.tsx`) and its own nav list.

From `nav.ts`, on why `ADMIN_NAV_ITEMS` is a separate list rather than an `adminOnly` flag: *"/admin is a standalone console behind its own sign-in, not a section of the product, and a flag would invite something to render these next to 'Generate' the moment someone forgot to filter on it."*

Admin capabilities

Capability	Endpoint
List / search / sort users	GET /api/admin/users
User detail	GET /api/admin/users/{id}
Suspend / unsuspend (incl. bulk)	POST /api/admin/users/{id}/suspend
Override plan	POST /api/admin/users/{id}/plan
Override usage counter	POST /api/admin/users/{id}/usage
Revoke all sessions	POST /api/admin/users/{id}/logout-all
Delete account	DELETE /api/admin/users/{id}
Platform overview + trends	GET /api/admin/overview
Provider / key health	GET /api/admin/system

`set_usage` exists as a support tool — *"your run failed, here's the credit back."* `set_suspended` notes that sessions are the caller's job: a suspended user holding a live token would otherwise keep working.

13 · Quota and Billing

Quota (services/quota.py, 184 lines)

Free accounts get `free_generations_per_month` (default **5**) generations per **UTC calendar month**. Pro is unmetered.

The lease pattern

Quota is **reserved before** the work starts and **refunded if it fails**, so a crashed run doesn't cost the user a credit.

Piece	Role
<code>current_period</code>	The billing bucket — UTC calendar month
<code>next_period_start</code>	Epoch seconds at which usage resets
<code>limit_for</code>	Allowance per month, or None for unlimited
<code>get_quota</code>	Read-only — consumes nothing
<code>QuotaLease</code>	A reserved unit. <code>.commit()</code> or <code>.refund()</code> .
<code>consume_quota</code>	Reserve one, or raise 402
<code>require_quota</code>	The dependency: authenticate, then reserve

At the store layer, `try_consume` is **atomic** — the reservation cannot double-spend under concurrency. `get_plan` resolves the *effective* plan: a lapsed paid plan reads as `free` without needing a background job to downgrade it.

Remaining quota is surfaced in the nav while signed in (`components/QuotaBar.tsx`).

Billing — Stripe (services/billing.py, 293 lines)

This module owns *what a plan costs* and *how a webhook is read*. It holds no Stripe SDK and no secret key — the checkout side is a hosted **Stripe Payment Link**, and the grant side is a signed webhook. Everything here is pure functions over payloads; the store writes live in `main.billing_webhook`.

The entitlement rule. An entitlement may only be granted by something that **proves a payment happened for a known user** — a hosted checkout plus a signed webhook. `set_plan`'s docstring is explicit: *"Call this from a verified payment webhook — never straight from a client request."*

Wiring it up

1. Create a Stripe Payment Link per plan (monthly, yearly) → `BILLING_CHECKOUT_URL_MONTHLY` / `_YEARLY`
2. Add a webhook endpoint at `/api/billing/webhook`, subscribed to `checkout.session.completed` and `invoice.paid`. Its signing secret (`whsec_...`) goes in `BILLING_WEBHOOK_SECRET`.
3. Nothing else.

Why `client_reference_id` carries both ids. A Payment Link accepts exactly one piece of caller state in its query string, echoed back on the Checkout Session. There is no arbitrary-metadata parameter, so the plan rides inside that single field alongside the user id — hence the `usr_x__monthly` encoding. Stripe restricts the value to alphanumerics, dashes and underscores, which is why the separator is `__` and not `:`.

Why renewals need the customer mapping. Only the Checkout Session echoes `client_reference_id`. The `invoice.paid` that arrives on every subsequent renewal identifies the payer solely by Stripe customer id, so the first payment must record `customer → user ( store.link_stripe_customer )` — *or every subscriber would lapse after one period.*

Function	Role
<code>plans</code>	The catalogue the pricing UI renders
<code>checkout_url</code>	Hosted Payment Link, tagged so the webhook can identify the payer
<code>verify_webhook</code>	Constant-time signature check over the raw body
<code>is_granting_event</code>	False for <code>payment.failed</code> , cancellations, refunds, and the many other webhook types that must not unlock anything
<code>extract_grant</code>	Pulls (<code>user_id</code> , <code>plan_id</code>) from a verified payload

Defaults are \$20/month, \$100/year.

14 · The Security Scanner

services/security_scanner.py — 926 lines

Given a URL the user owns, the scanner fetches the site and inspects its **configuration**. It is explicitly non-intrusive: it reads headers, cookies, TLS/redirect behaviour, and probes a short curated list of files that should never be public. **It sends no attack payloads and brute-forces nothing.**

Checks performed

Check	Covers
<code>_check_transport</code>	HTTPS enforcement, HSTS
<code>_check_security_headers</code>	CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy
<code>_check_cookies</code>	Secure, HttpOnly, SameSite — reported per <i>problem</i> , not per cookie
<code>_check_disclosure</code>	Server / framework version leakage
<code>_check_cors</code>	CORS misconfiguration
<code>_check_content</code>	Content-level issues
<code>_check_exposed_files</code>	<code>.env</code> , <code>.git/config</code> , and similar

SSRF defence, in two layers.

`_assert_public_host` resolves the hostname and rejects private, loopback, link-local, reserved, and multicast IPs.

`_redirect_guard` re-validates **every redirect hop** as an httpx response hook, firing *before* each redirect is followed — so an open redirect on the target cannot bounce the scanner onto an internal address. Validating only the initial URL would leave exactly that hole.

This same guarded path is what DOM grounding (§6) reuses to fetch a user's deployed page.

Scoring

Findings roll into a 0–100 score and a letter grade. Penalties are **capped per category**, because a plain sum let one repeated misconfiguration sink everything: *"five cookies each missing three flags is -120 on a 100-point scale, which graded any site with a few analytics cookies F no matter how good the rest of it was."*

`exposure` is uncapped on purpose: *"a world-readable .env is not one issue among several, and it should be able to take the grade to F by itself."*

`_looks_sensitive` confirms a 200 response actually resembles the sensitive file rather than an SPA catch-all returning 200 for everything — the difference between a real finding and a false positive on every single-page app.

`_youtube` generates a YouTube *search* URL for each fix, never a specific video — a link that can't rot into someone else's content.

15 · GitHub Integration and Publishing

Fetching (services/github_service.py, 230 lines)

Path	Mechanism
Public repos	raw.githubusercontent.com — no auth, no rate limits, faster
Private repos	GitHub REST Contents API with OAuth token or PAT
ZIP upload	file_extractor.extract_zip

`get_file_tree` uses the Git Trees API with `recursive=1` — the whole tree in one request.
`parse_github_url` handles bare repo URLs, `/tree/branch`, and `/tree/branch/subpath`, and its test suite pins tricky cases: `krishna.github.io`, `awesome.gitignore-demo`, `my.github.io.git` — i.e. stripping `.git` without mangling names that legitimately contain it.

Errors: `RepoNotFoundError`, `RepoAccessError`.

Publishing (services/git_publisher.py, 265 lines)

Creates a fresh GitHub repo and pushes a whole project into it in a **single commit**, using the Git Data API end-to-end — **no local `git` binary required**.

```
1. GET    /user                -> resolve authenticated owner
2. POST   /user/repos (auto_init) -> create repo with a base commit
3. GET    /git/ref/heads/{branch} -> base commit sha
4. GET    /git/commits/{sha}      -> base tree sha
5. POST   /git/blobs (per file)   -> blob shas, uploaded concurrently
6. POST   /git/trees             -> new tree layered on the base
7. POST   /git/commits          -> new commit
8. PATCH  /git/refs/heads/{branch} -> move the ref
```

Publish-then-generate ordering. `main.py` notes that tests are generated and validated locally — through the writer agent's self-heal loop and the grounding pipeline — *before* the push. The repo is only created once there is something worth pushing.

`was_published_by` is the authorization gate on deletion: **true only if THIS user published THIS repo through the app**. Testra will not delete a repo it did not create on that user's behalf, even with a valid token that could.
`delete_repo` is marked irreversible.

16 · The Data Model

services/store.py — 1,157 lines. Stdlib `sqlite3` only; thread-safe; no ORM.

Tables

Table	Columns
users	id, email, password_hash, name, github_id, github_login, avatar_url, created_at, plan, plan_expires_at, email_verified, suspended, suspended_at, suspended_reason, stripe_customer_id , google_id
sessions	session_id, created_at, expires_at, data
jobs	job_id, created_at, updated_at, data
usage	user_id, period, count
generations	id, user_id, created_at, source, framework, language, test_count, file_count, status, duration_ms, error
scans	id, user_id, created_at, url, grade, score, findings, duration_ms, counts
published_repos	full_name, user_id, repo_url, created_at, test_count, files_pushed, cicd_added, private
user_settings	user_id, framework, language, base_url, updated_at

Indexed on `users(stripe_customer_id)` and `users(google_id)` — both created *after* `_migrate` runs, because both columns arrive by ALTER and do not exist on a deployed database until then.

Migrations

`CREATE TABLE IF NOT EXISTS` silently skips a table that's already there, so new columns need an explicit ALTER against the live schema. `_ADDED_COLUMNS` drives this. Every column must be nullable or carry a DEFAULT — SQLite cannot add a NOT NULL column without one to a table that already has rows.

Two defaults chosen for the data that already exists, not for the ideal:

- `email_verified` defaults to `1`, not `0`: accounts predating email verification were created when the address was never checked, and flipping them to unverified *would lock out every existing user at once*. New rows pass `email_verified=0` explicitly.
- `generations.status` defaults to `'success'` because that is what every existing row is — until failure recording existed, a generation was only written after it had already succeeded.

Lifecycle management

- `_migrate` — adds columns to tables that already exist in deployed databases
- `prune` / `prune_sessions` — TTL expiry (`job_ttl_seconds` , default 24h)
- `vacuum` — reclaims file space, *"SQLite keeps pages otherwise"*
- `sweep` — expire stale jobs and sessions, then reclaim

Notable method semantics

Method	Why it's written the way it is
activity_totals	Lifetime counts, deliberately separate from the capped list_* methods — <i>"the headline number must not silently"</i> track a display cap
activity_feed	Every feature's activity as one timeline, newest first
activity_series	Per-day counts, zero-filled
platform_series	The all-tenant twin of activity_series, zero-filled the same way
_escape_like	Neutralises LIKE wildcards so a search term matches itself literally
try_consume	Atomic quota reservation
get_plan	Effective plan — lapsed paid reads as free with no background job
link_stripe_customer	Maps Stripe customer → user so renewals can be attributed
set_suspended	Reversible admin lock; keeps the audit trail queryable
delete_account	Erases a user and everything belonging to them. Irreversible.
recent_failures	Failed generations across all users — <i>"the admin's first stop when something is broken"</i>

17 · Complete API Reference

45 routes. All under `main.py` (2,116 lines).

System

Method	Path	Purpose
GET	/health	Liveness
GET	/metrics	Metrics
GET	/api/status	Provider health for the public status panel
GET	/api/logs	Recent router call log

Generation pipeline

Method	Path	Guards
POST	/api/analyze	analyze_limiter · NDJSON
POST	/api/upload-zip	analyze_limiter · NDJSON
POST	/api/generate/{job_id}	generate_limiter · require_quota + ownership
GET	/api/stream/{job_id}	generate_limiter · SSE

Authentication

Method	Path	Purpose
GET	/api/auth/config	Which auth methods this server offers
POST	/api/auth/signup	Register
POST	/api/auth/login	Password check → session or OTP challenge
POST	/api/auth/login/verify-otp	Complete OTP login
POST	/api/auth/verify-email	Redeem a verification link
POST	/api/auth/resend-verification	email_limiter
POST	/api/auth/forgot-password	email_limiter
POST	/api/auth/reset-password	Redeem a reset token
POST	/api/auth/change-password	Authenticated change
POST	/api/auth/logout	End this session
POST	/api/auth/logout-all	Revoke every session
GET	/api/auth/github/login · /callback	GitHub OAuth
GET	/api/auth/google/login · /callback	Google OAuth
GET	/api/auth/me	Current user

Account

Method	Path	Purpose
GET	/api/profile	Profile + lifetime activity
GET	/api/dashboard	Feed, series, published repos
GET / PUT	/api/settings	Form defaults
DELETE	/api/account	Irreversible erase

Admin

Method	Path
GET	/api/admin/users · /api/admin/users/{id}
POST	/api/admin/users/{id}/suspend · /plan · /usage · /logout-all
DELETE	/api/admin/users/{id}
GET	/api/admin/overview · /api/admin/system

Billing

Method	Path	Note
GET	/api/billing/me	Plan + quota state
POST	/api/billing/checkout	Hosted Stripe Payment Link
POST	/api/billing/webhook	The only path that grants entitlement. Signature-verified.

Publishing and scanning

Method	Path	Note
POST	/api/publish-zip	Create repo + push · NDJSON
GET	/api/repos	Repos published through Testra
DELETE	/api/repo/{owner}/{repo}	Gated by was_published_by
POST	/api/scan	Passive security scan · NDJSON

18 · Frontend Architecture

React 18 · TypeScript · Vite · React Router · 61 files, ~9,000 lines

Routes

Path	Page	Access
/	Home	Public
/how-it-works	HowItWorks	Public
/login · /signup	Login / Signup	Public
/auth/callback	AuthCallback	Public
/verify-email	VerifyEmail	Public
/forgot-password · /reset-password	ForgotPassword / ResetPassword	Public
/status	Status	Public
/dashboard	Dashboard	Protected
/generate	Generate	Protected
/publish	Publish	Protected
/scan	Scan	Protected
/settings · /profile	Settings / Profile	Protected
/admin · /admin/users · /admin/system	Admin console	Admin
*	NotFound	Public

State and access control

- `context/AuthContext.tsx` — the single source of session state
- `components/ProtectedRoute.tsx` — redirects unauthenticated users
- `components/ErrorBoundary.tsx` — catches render failures
- `nav.ts` — `NAV_ITEMS` (product) and `ADMIN_NAV_ITEMS` (console), defined once

One nav list, defined once. The sidebar renders `NAV_ITEMS` on every signed-in page, so `NavBar` no longer carries its own copy — *“two lists that had to be kept in step was exactly how `/profile` ended up reachable from one menu and not the other.”*

Client-side gating is cosmetic by design. `user.is_admin` decides only whether the nav item renders. Every admin route re-derives the role server-side. The frontend check is UX; the server check is security.

Component groups (29 components)

Group	Components
Generation flow	GenerateInput, ConfigureStep, PreviewStep, ResultsStep, StepIndicator, StreamingOutput, CodeViewer
Progress & trust	FlowPipeline, TrustPanel
Dashboard	DashboardLayout, ActivityChart, CountUp, QuotaBar
Admin	AdminLayout, AdminLogin, AdminOverview, AdminUsers (775 lines), AdminSystem
Auth	AuthForm, ProtectedRoute
Features	PublishPanel, ScanPanel, ProviderStatus, UpgradeModal
Presentation	Hero3D, AuroraBackground, TiltCard, ScrollProgress, Page, PageHeader, Footer, NavBar, ErrorBoundary

Utilities

- `utils/api.ts` (842 lines) — the entire typed API client, including NDJSON stream parsing and `raiseApiError`
- `types/index.ts` (339 lines) — shared types mirroring the backend schemas
- `flows.ts` (272 lines) — the step-id contract with `services/progress.py`
- `utils/format.ts` , `utils/download.ts` , `utils/chartPalette.ts` , `lib/celebrate.ts`

Build output

`npm run build` runs `tsc && vite build`. Current output:

Asset	Raw	Gzipped
<code>index.js</code>	666.87 kB	198.30 kB
<code>Hero3D.js</code>	828.27 kB	223.70 kB
<code>index.css</code>	39.75 kB	8.14 kB

2,577 modules transformed. Both JS chunks exceed Vite's 500 kB warning threshold — the 3D hero is the dominant cost and remains a code-splitting candidate (§23).

19 · Configuration Reference

config.py — 191 lines, typed via `pydantic-settings`. Read from the environment and `backend/.env` in one place, rather than scattered `os.getenv` calls. 38 settings.

Application

Setting	Default
app_env	development
log_level	INFO
cors_origins	*
frontend_url	http://localhost:5173
backend_url	http://localhost:8000

Sessions and OAuth

Setting	Default
github_client_id/ github_client_secret	empty
google_client_id/ google_client_secret	empty
oauth_session_ttl_seconds	28,800 (8h)
session_secret	empty — tokens stored unencrypted if unset (local dev only)

Email

Setting	Default
smtp_host / smtp_user / smtp_password / smtp_from	empty
smtp_port	587
smtp_from_name	Testra
smtp_starttls / smtp_ssl	true / false
smtp_timeout	15
require_email_verification	true
login_otp_enabled	true
email_verify_ttl_seconds	86,400 (24h)
password_reset_ttl_seconds	3,600 (1h)
login_otp_ttl_seconds	600 (10m)
login_otp_max_attempts	5

Limits and pipeline

Setting	Default
max_repo_size_mb	100
max_files_per_repo	2,000
token_budget	600,000 — see §23
job_ttl_seconds	86,400 (24h)

Plans, billing, admin

Setting	Default
free_generations_per_month	5
pro_price_monthly_usd / _yearly_usd	20 / 100
billing_checkout_url_monthly / _yearly	empty
billing_webhook_secret	empty
billing_contact_email	empty
admin_emails	empty — the entire admin grant

Derived properties

is_production , cors_origin_list , admin_email_list , is_admin_email , admin_enabled , github_oauth_enabled , google_oauth_enabled , smtp_configured , mail_from_address , email_enabled , billing_enabled , oauth_callback_url . Admin emails are compared lowercased, because that is how users.email is stored.

API keys (GEMINI_API_KEY_1..10 etc.) are read directly from the environment by llm_router , not through Settings — they are variable in count.

20 · Testing and CI

Suite

650 tests across 22 files.

File	Lines	Covers
test_security_scanner.py	658	Checks, scoring caps, SSRF guards
test_scaffold.py	497	Manifests, framework config, CI, suite directory
test_admin.py	442	Admin grant, verification requirement, 404-not-403
test_file_extractor.py	434	Filtering, scoring, framework detection
test_billing.py	378	Stripe webhook verification, grant extraction, renewals
test_progress.py	344	NDJSON framing, pre-flight vs in-band errors, step ids
test_verification.py	339	Tokens, OTP, expiry, attempt caps
test_store.py	332	Persistence, migrations, pruning
test_llm_router.py	330	Failover, rotation, cooldown, tiers, provenance
test_quota.py	192	Leases, refunds, period rollover
test_grounding.py	157	Source and DOM anchor extraction, membership
test_google_oauth.py	139	Authorize URL, code exchange, userinfo
test_git_publisher.py	137	Blob → tree → commit → ref flow
test_fragility.py	136	Feature weights, scoring, grade boundaries
test_filter_agent.py	132	Agent 1 behaviour
test_importance_model.py	120	Fit, predict, safe fallback with no artifact
test_writer_agent.py	113	Selector extraction/validation, JSON parsing
test_validator.py	111	Syntax checks, verified-vs-unchecked accounting
test_benchmark.py	98	Rates, denominators, leaderboard
test_auth.py	90	Hashing, validation, token extraction
test_github_service.py	69	URL parsing edge cases
test_ratelimit.py	24	Sliding window

5,272 lines of tests. The admin and verification tests import `services.admin` / `services.verification` directly rather than through `main.py`, so they exercise the modules without standing up the app.

CI (.github/workflows/ci.yml)

Job	Runs
Backend	Matrix Python 3.11 / 3.12 / 3.13 · ruff check . · pytest tests/ -v --cov=. --cov-report=term-missing
Frontend	Node 20 · npm run build (tsc + vite)
Deploy	master only, after both jobs pass, pushes only — see §21

Triggers: push to `master` / `main` / `develop` ; PRs to `master` / `main` . Default permissions are `contents: read` — the deploy job widens them for itself, and only for the OIDC token it needs.

21 · Deployment NEW

deploy/ · .github/workflows/ci.yml · docs/DEPLOY.md

The target is a **single EC2 instance** running Docker Compose behind Caddy, rolled forward by GitHub Actions over SSM. There is no Kubernetes, no load balancer, and no container orchestrator — the same "no infrastructure to provision" principle as §2.

The rollout

1. **Assume the deploy role via OIDC** — *the pipeline holds no long-lived AWS keys at all.* `id-token: write` is granted only to this job.
2. **Build and push two images to ECR** — backend and web, each tagged with the commit SHA.
3. **Roll the instance** via `aws ssm send-command`: checkout the SHA, log in to ECR, `docker compose pull + up -d --remove-orphans`, then `docker image prune -f`.
4. **Verify the site actually answers** — poll `/health` for a 200.

Decisions recorded in the workflow, each with its failure mode:

- **Never** `:latest`. Images are tagged with the SHA — *"a box restarting must come back on the exact build it was on."*
- **Never on** `pull_request`. *"A fork's PR must not be able to reach AWS."*
- **Concurrency group, cancel-in-progress: false**. Two deploys racing would leave the box on whichever finished last. Queue them; never cancel one mid-roll.
- **Secrets are not passed from CI**. The app reads them from `/var/data/testra.env`, which only the box can see.
- **Not** `aws ssm wait`. Its 100s ceiling is shorter than an image pull, so the workflow polls `get-command-invocation` itself.
- **Image pruning**. Old layers accumulate until the volume fills, and *SQLite on a full disk fails writes*.
- **VITE_API_URL is a build input, not a runtime one**. Vite inlines it into the bundle — *"wrong value here ships a broken frontend."*

A green SSM command only means the containers started. The health-check step is what notices the app came up broken. There is **no automatic rollback**: on failure the workflow says so explicitly and tells you to redeploy the previous good SHA from the Actions tab.

The instance itself is provisioned by `deploy/aws-setup.sh` and `deploy/bootstrap.sh`; the runtime topology is `deploy/docker-compose.prod.yml`. Caddy terminates TLS and proxies to uvicorn — which is why the backend runs with `--proxy-headers` (§10). The OIDC trust policy is pinned to the repository's current name.

22 · Known Limitations

From `docs/LIMITATIONS.md` — the project's own model card. Reproduced because it is load-bearing for anyone trusting the output.

What "validated" means — and doesn't. The green *"passed a syntax check"* badge means each generated file **parses**. It does **not** mean the tests were executed, or that assertions pass. Selector *resolution* is a separate and stronger claim, made only where §6's grounding says so — and even a fully grounded suite has never been run. Always run the suite against your app before trusting it in CI.

Limitation	Detail
Selector hallucination	Extraction is regex-based today. It can miss dynamic classes (<code>className={styles.x}</code>) and CSS-in-JS — so both false positives and false negatives are possible. Grounding narrows this but does not close it.
No execution	Nothing runs the generated suite. Grounding proves selectors resolve; it cannot prove a flow passes.
Large repos	May exceed the model's context window. Low-priority files are summarized rather than sent verbatim, which reduces selector accuracy.
Output size	Extremely large suites can hit provider output-token limits. Narrow the scope and regenerate.
Free-tier variance	Quality and latency depend on whichever provider is available. Results can vary run to run — the provenance report says which model answered.
Importance model	Ships untrained. Until an artifact exists it is a no-op and the frozen heuristics apply.

Data handling

- Repo contents are held only for a job's lifetime (SQLite, pruned after 24h) and sent to the selected LLM provider
- Linked GitHub tokens are encrypted at rest **when** `SESSION_SECRET` is set
- The scanner is passive and refuses internal targets, including on redirect hops — and DOM grounding inherits that guard

Operational limits not in the model card

- **Rate limiting is per-worker.** In-memory windows mean N workers permit roughly N× the intended rate. `ratelimit.py` flags Redis as the production fix. (Separately, the limiters only see real client IPs because of the proxy-header configuration in §10.)
- **Admin changes need a restart** — an accepted trade, not a bug (§12).
- **No automatic rollback** on a failed deploy (§21).

Stated roadmap

- Execute generated suites in a sandbox and report real pass/fail

- AST-based selector extraction (roles, accessible names, visible text)
- Retrieval + per-provider context budgeting for large repos (see §23.1)
- Re-fit the fragility weights and importance model on real outcomes

23 · Open Findings

UNRESOLVED

Identified during review of the generation pipeline. These are not in the project's own docs. Estimates are from code reading, not measurement. Line references verified against the current tree.

23.1 — The token budget exceeds two of three providers' context windows

Severity: high. This is a correctness issue, not an optimization.

`token_budget` defaults to **600,000** (`config.py:89` , `file_extractor.py:351`). The router's fallback chain is:

Provider	Context	Fits 600k?
Gemini 2.0 Flash	~1M	Yes
Groq llama-3.3-70b	128k	No
Claude Haiku 4.5 / Opus 4.8	200k	No

Each `ModelSpec` caps output via `max_output_tokens` . **There is no equivalent input guard anywhere.** The budget was sized for Gemini — the source comment at `llm_router.py:213` still says so (*# 1M context swallows big repos*) — but nothing prevents that same oversized prompt reaching a fallback that cannot hold it.

Consequence: on a repo near the budget, Gemini succeeds and both fallbacks fail on context length. The rotation that exists specifically to survive rate limits cannot fall back. Worse, `main.py` catches the resulting `AllProvidersExhausted` and reports it as *"our shared API keys are dry"* — an outage message for what is actually a prompt-length problem.

Now compounded by tiers: a Pro request tries Claude *first*. On a large repo the paying user hits the 200k wall before the 1M-context provider is ever attempted, so the upgrade makes large-repo failures *more* likely, not less.

Compounding: the estimator is `len(content) // 4` , which undercounts code — so true token counts run *above* the budget's own accounting.

Suggested fix: a per-provider input budget alongside `max_output_tokens` , and a distinct error for "prompt too large for every remaining provider" so it stops being misreported as an outage.

23.2 — The full repo source is re-sent once per generated file

Severity: high (cost/latency).

`writer_agent.py:921` embeds `json.dumps(filter_result.files, indent=2)` — the entire extracted source — into every single-file prompt. That prompt is built inside the `for spec in plan:` loop (`writer_agent.py:800`), and `_plan_files` returns up to 12 files (`writer_agent.py:872`). The full source is therefore sent **up to 12 times per generation**, byte-identical each time.

Suggested fix: have `_plan_files` return the source paths each planned file actually depends on, and send only those. A page object for `LoginPage` does not need every component in the repo. Estimated 5–8× reduction in input tokens, on every provider — and it brings prompts back inside the fallback context windows in 23.1.

23.3 — Prompt ordering makes caching impossible

Severity: medium (cost only).

Prompt caching is a prefix match. In `_generate_one_file (writer_agent.py:877)`, the only content that varies across the 12 calls is `spec['filename']` and its description — and it sits at the **top** of the prompt (`## FILE TO WRITE , writer_agent.py:897`). Everything after it is identical. Because the one volatile field is at position zero, there is no reusable prefix at all.

Suggested fix: move that block to the end, then add a `cache_control: {type: "ephemeral"}` breakpoint. No `cache_control` is set anywhere in the codebase today. The source block clears the minimum cacheable prefix easily.

Caching reduces cost, not tokens. The same tokens are still sent and counted — only the price of the repeated prefix changes. 23.2 is the provider-agnostic fix; this one compounds with it. Note that under the Pro tier Claude is now *first* in rotation, so a cache would pay off far more often than it would have before.

23.4 — Minor

- **Redundant payload.** `_extract_available_selectors (writer_agent.py:1009)` already distills selectors from the source, yet the prompt sends *both* the distilled list and the full raw source.
- **Wasted whitespace.** `indent=2` on the embedded JSON blobs (`writer_agent.py:901, 907, 921`) adds pure formatting tokens the model gains nothing from.
- **Docstring/behaviour drift — filter agent.** `filter_agent._build_file_tree_summary` claims to show structure "without sending all file contents", but `any(hp in path for hp in ["pages/", "router"])` (`filter_agent.py:263`) matches *every* file under `pages/` , each contributing an 800-char preview. Bounded per file, unbounded in file count.
- **Docstring/behaviour drift — router.** The `llm_router.py` module header still describes a fixed Gemini → Groq → Claude order and claims "*All providers are FREE tier — no credit card required.*" Both predate the tier system (\$8) and are wrong for Pro, which leads with Opus.
- **Bundle size.** The `Hero3D` chunk is 828.27 kB (223.70 kB gzipped) and the main chunk has grown to 666.87 kB (198.30 kB); both trip Vite's 500 kB warning.

24 · Appendix — File Inventory

Backend — 32 files, ~12,000 lines (excluding tests)

File	Lines	Role
main.py	2116	FastAPI app, 45 routes
services/store.py	1157	SQLite persistence, 8 tables
agents/writer_agent.py	1113	Agent 2 + self-heal loop
services/llm_router.py	948	Tiered provider rotation
agents/scaffold.py	933	Runnable suite skeleton
services/security_scanner.py	926	Passive scan + SSRF guard
services/file_extractor.py	658	Deterministic pre-filter
models/schemas.py	418	Pydantic request/response models
services/grounding.py	349	Source + DOM selector verification
services/billing.py	293	Stripe plans, entitlement
agents/filter_agent.py	269	Agent 1
services/git_publisher.py	265	Git Data API push
services/fragility.py	262	Break-risk scoring
services/verification.py	253	One-time secrets
services/github_service.py	230	Repo fetching
services/importance_model.py	226	Learned file scorer
services/auth.py	208	Sessions, hashing, dependency
services/benchmark.py	198	Quality harness
services/progress.py	191	NDJSON step progress
config.py	191	Typed settings
services/quota.py	184	Per-user metering
services/validator.py	162	Syntax checks
services/mailer.py	112	SMTP transport
services/google_oauth.py	108	Google OIDC handshake
services/github_oauth.py	93	GitHub OAuth handshake

ratelimit.py	57	Sliding windows
services/admin.py	57	The admin role
logging_config.py	37	JSON logs + request IDs
tests/*	5272	22 files, 650 tests

Frontend — 61 files, ~9,000 lines

File	Lines
utils/api.ts	842
pages/AdminUsers.tsx	775
pages/Settings.tsx	464
pages/Profile.tsx	388
types/index.ts	339
pages/Dashboard.tsx	293
components/AuthForm.tsx	282
flows.ts	272
pages/AdminOverview.tsx · AdminSystem.tsx	271 · 270
components/TrustPanel.tsx	266
components/PublishPanel.tsx	235
pages/AdminLogin.tsx · components/ResultsStep.tsx	234 · 234
pages/HowItWorks.tsx	217
components/ScanPanel.tsx · ActivityChart.tsx	214 · 214
components/ConfigureStep.tsx	206
components/UpgradeModal.tsx	205
components/FlowPipeline.tsx	195
Remaining 41 files	<165 each

Repository layout

```
testgen-ai/
├── backend/
│   ├── agents/          filter_agent · writer_agent · scaffold
│   ├── models/          schemas
│   ├── services/        20 modules
│   ├── tests/           22 files, 650 tests
│   ├── main.py · config.py · ratelimit.py · logging_config.py
│   ├── Dockerfile       (uvicorn --proxy-headers)
│   └── requirements.txt · .env.example
├── frontend/
│   ├── src/ components/ · pages/ · context/ · utils/ · lib/ · types/
│   └── Dockerfile.prod · Caddyfile
├── deploy/              aws-setup.sh · bootstrap.sh · docker-compose.prod.yml
├── docs/                ARCHITECTURE.md · DEPLOY.md · LIMITATIONS.md
│   └── Testra-Complete-Reference.html/.pdf
├── .github/workflows/   ci.yml (backend · frontend · deploy)
├── docker-compose.yml
└── README.md
```

Regenerate this document with `docs/build-reference.sh`, which renders `Testra-Complete-Reference.html` to PDF via headless Chrome.